Ex no: 1	Linked List
Date:	

Question 1:

Aim:

To develop a C program to create and manage an audio playlist using linked list data structure. It helps to perform operations like adding, deleting and displaying songs dynamically.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Song {
    char title[100];
    struct Song *next;
} Song;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Song *create_song(const char *title) {
    Song *song = (Song *) malloc(sizeof(Song));
    if (song == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(song->title, title, sizeof(song->title) - 1);
    song->title[sizeof(song->title) - 1] = '\0';
    song->next = NULL;
    return song;
}

static void add_song(Song **head, const char *title) {
    Song *song = create_song(title);
    if (*head == NULL) {
        *head = song;
        return;
    }

    Song *temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = song;
```

```
}

static void insert_after(Song *head, const char *after_title, const char *new_title) {
    Song *temp = head;
    while (temp != NULL && strcmp(temp->title, after_title) != 0) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Song not found in playlist.\n");
        return;
    }

    Song *song = create_song(new_title);
    song->next = temp->next;
    temp->next = song;
}

static void delete_song(Song **head, const char *title) {
    Song *temp = *head;
    Song *prev = NULL;

    while (temp != NULL && strcmp(temp->title, title) != 0) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Song not found.\n");
        return;
    }

    if (prev == NULL) {
        *head = temp->next;
    } else {
        prev->next = temp->next;
    }
    free(temp);
    printf("Song removed from playlist.\n");
}

static void display_playlist(Song *head) {
    if (head == NULL) {
        printf("Playlist is empty.\n");
        return;
    }

    printf("\nPlaylist:\n");
    int index = 1;
    while (head != NULL) {
        printf("%d. %s\n", index++, head->title);
        head = head->next;
    }
}
```

```
static void free_playlist(Song *head) {
    while (head != NULL) {
        Song *next = head->next;
        free(head);
        head = next;
    }
}

int main(void) {
    Song *playlist = NULL;
    int choice;
    char title[100];
    char after_title[100];

    while (1) {
        printf("\nAudio Player Playlist Menu\n");
        printf("1. Add song\n");
        printf("2. Insert song after another song\n");
        printf("3. Delete song\n");
        printf("4. Display playlist\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter song title: ");
                read_line(title, sizeof(title));
                add_song(&playlist, title);
                break;
            case 2:
                printf("Enter existing song title: ");
                read_line(after_title, sizeof(after_title));
                printf("Enter new song title: ");
                read_line(title, sizeof(title));
                insert_after(playlist, after_title, title);
                break;
            case 3:
                printf("Enter song title to delete: ");
                read_line(title, sizeof(title));
                delete_song(&playlist, title);
                break;
            case 4:
                display_playlist(playlist);
                break;
            case 5:
                free_playlist(playlist);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }
```

```
    free_playlist(playlist);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Assignment$ cd Ass_2_ds
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./1_audio_player_playlist_linked_list

Audio Player Playlist Menu
1. Add song
2. Insert song after another song
3. Delete song
4. Display playlist
5. Exit
Enter your choice: 1
Enter song title: Vibe_Aavoma

Audio Player Playlist Menu
1. Add song
2. Insert song after another song
3. Delete song
4. Display playlist
5. Exit
Enter your choice: 1
Enter song title: Yathe Yathe

Audio Player Playlist Menu
1. Add song
2. Insert song after another song
3. Delete song
4. Display playlist
5. Exit
Enter your choice: 2
Enter existing song title: Yathe Yathe
Enter new song title: Pavalamali

Audio Player Playlist Menu
1. Add song
2. Insert song after another song
3. Delete song
4. Display playlist
5. Exit
Enter your choice: 4

Playlist:
1. Vibe_Aavoma
2. Yathe Yathe
3. Pavalamali
```

```
Audio Player Playlist Menu
1. Add song
2. Insert song after another song
3. Delete song
4. Display playlist
5. Exit
Enter your choice: 3
Enter song title to delete: Pavalamali
Song removed from playlist.

Audio Player Playlist Menu
1. Add song
2. Insert song after another song
3. Delete song
4. Display playlist
5. Exit
Enter your choice: 4

Playlist:
1. Vibe_Aavoma
2. Yathe Yathe

Audio Player Playlist Menu
1. Add song
2. Insert song after another song
3. Delete song
4. Display playlist
5. Exit
Enter your choice: 5
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the C program for audio playlist management was compiled and executed successfully. The playlist operations were performed correctly and the output was verified.

Question 2:

Write a C program to create a Congo line using linked list

Aim:

To write a C program to simulate the formation of a congo line using linked list. This program demonstrates dynamic insertion and traversal of nodes.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Person {
    char name[100];
    struct Person *next;
} Person;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Person *create_person(const char *name) {
    Person *person = (Person *) malloc(sizeof(Person));
    if (person == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(person->name, name, sizeof(person->name) - 1);
    person->name[sizeof(person->name) - 1] = '\0';
    person->next = NULL;
    return person;
}

static void join_end(Person **head, const char *name) {
    Person *person = create_person(name);
    if (*head == NULL) {
        *head = person;
        return;
    }

    Person *temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = person;
}

static void join_front(Person **head, const char *name) {
    Person *person = create_person(name);
    person->next = *head;
    *head = person;
```

```
}

static void remove_person(Person **head, const char *name) {
    Person *temp = *head;
    Person *prev = NULL;

    while (temp != NULL && strcmp(temp->name, name) != 0) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Person not found in the congo line.\n");
        return;
    }

    if (prev == NULL) {
        *head = temp->next;
    } else {
        prev->next = temp->next;
    }
    free(temp);
}

static void display_line(Person *head) {
    if (head == NULL) {
        printf("The congo line is empty.\n");
        return;
    }

    printf("\nCongo line order:\n");
    while (head != NULL) {
        printf("%s", head->name);
        head = head->next;
        if (head != NULL) {
            printf(" -> ");
        }
    }
    printf("\n");
}

static void free_line(Person *head) {
    while (head != NULL) {
        Person *next = head->next;
        free(head);
        head = next;
    }
}

int main(void) {
    Person *head = NULL;
    int choice;
    char name[100];

    while (1) {
```

```
        printf("\nCongo Line Menu\n");
        printf("1. Join at end\n");
        printf("2. Join at front\n");
        printf("3. Remove person\n");
        printf("4. Display line\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter person name: ");
                read_line(name, sizeof(name));
                join_end(&head, name);
                break;
            case 2:
                printf("Enter person name: ");
                read_line(name, sizeof(name));
                join_front(&head, name);
                break;
            case 3:
                printf("Enter name to remove: ");
                read_line(name, sizeof(name));
                remove_person(&head, name);
                break;
            case 4:
                display_line(head);
                break;
            case 5:
                free_line(head);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_line(head);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Assignment$ cd Ass_2_ds
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./2_congo_line_linked_list

Congo Line Menu
1. Join at end
2. Join at front
3. Remove person
4. Display line
5. Exit
Enter your choice: 1
Enter person name: Ashwin S

Congo Line Menu
1. Join at end
2. Join at front
3. Remove person
4. Display line
5. Exit
Enter your choice: 2
Enter person name: Ash S

Congo Line Menu
1. Join at end
2. Join at front
3. Remove person
4. Display line
5. Exit
Enter your choice: 3
Enter name to remove: Ash S

Congo Line Menu
1. Join at end
2. Join at front
3. Remove person
4. Display line
5. Exit
Enter your choice: 4

Congo line order:
Ashwin S
```

```
Congo Line Menu
1. Join at end
2. Join at front
3. Remove person
4. Display line
5. Exit
Enter your choice: 4

Congo line order:
Ashwin S

Congo Line Menu
1. Join at end
2. Join at front
3. Remove person
4. Display line
5. Exit
Enter your choice: 5
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the congo line simulation program was executed successfully. The members were arranged and displayed in sequence and the output was verified.

Question 3:

Write a C program to implement scavenger hunt or treasure hunt using linked list.

Aim:

To implement a scavenger hunt game by storing and accessing clues using linked list. This helps to understand sequential data access.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Clue {
    char text[200];
    int completed;
    struct Clue *next;
} Clue;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Clue *create_clue(const char *text) {
    Clue *clue = (Clue *) malloc(sizeof(Clue));
    if (clue == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(clue->text, text, sizeof(clue->text) - 1);
    clue->text[sizeof(clue->text) - 1] = '\0';
    clue->completed = 0;
    clue->next = NULL;
    return clue;
}

static void add_clue(Clue **head, const char *text) {
    Clue *clue = create_clue(text);
    if (*head == NULL) {
        *head = clue;
        return;
    }

    Clue *temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = clue;
}

static void mark_next_completed(Clue *head) {
    while (head != NULL && head->completed) {
```

```
        head = head->next;
    }

    if (head == NULL) {
        printf("All clues are already completed.\n");
        return;
    }

    head->completed = 1;
    printf("Completed clue: %s\n", head->text);
}

static void display_clues(Clue *head) {
    if (head == NULL) {
        printf("No clues available.\n");
        return;
    }

    int index = 1;
    while (head != NULL) {
        printf("%d. [%s] %s\n", index++, head->completed ? "Done" : "Pending", head->text);
        head = head->next;
    }
}

static void find_clue(Clue *head, const char *keyword) {
    int found = 0;
    while (head != NULL) {
        if (strstr(head->text, keyword) != NULL) {
            printf("Found clue: %s\n", head->text);
            found = 1;
        }
        head = head->next;
    }

    if (!found) {
        printf("No clue matched the keyword.\n");
    }
}

static void free_clues(Clue *head) {
    while (head != NULL) {
        Clue *next = head->next;
        free(head);
        head = next;
    }
}

int main(void) {
    Clue *head = NULL;
    int choice;
    char text[200];

    while (1) {
        printf("\nScavenger Hunt Menu\n");
```

```
        printf("1. Add clue\n");
        printf("2. Mark next clue as completed\n");
        printf("3. Display all clues\n");
        printf("4. Search clue by keyword\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter clue text: ");
                read_line(text, sizeof(text));
                add_clue(&head, text);
                break;
            case 2:
                mark_next_completed(head);
                break;
            case 3:
                display_clues(head);
                break;
            case 4:
                printf("Enter keyword: ");
                read_line(text, sizeof(text));
                find_clue(head, text);
                break;
            case 5:
                free_clues(head);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_clues(head);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Assignment$ cd Ass_2_ds
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./3_scavenger_hunt_linked_list

Scavenger Hunt Menu
1. Add clue
2. Mark next clue as completed
3. Display all clues
4. Search clue by keyword
5. Exit
Enter your choice: 1
Enter clue text: Lap

Scavenger Hunt Menu
1. Add clue
2. Mark next clue as completed
3. Display all clues
4. Search clue by keyword
5. Exit
Enter your choice: 2
Completed clue: Lap

Scavenger Hunt Menu
1. Add clue
2. Mark next clue as completed
3. Display all clues
4. Search clue by keyword
5. Exit
Enter your choice: 1
Enter clue text: top

Scavenger Hunt Menu
1. Add clue
2. Mark next clue as completed
3. Display all clues
4. Search clue by keyword
5. Exit
Enter your choice: 3
1. [Done] Lap
2. [Pending] top
```

```
Scavenger Hunt Menu
1. Add clue
2. Mark next clue as completed
3. Display all clues
4. Search clue by keyword
5. Exit
Enter your choice: 4
Enter keyword: l
No clue matched the keyword.

Scavenger Hunt Menu
1. Add clue
2. Mark next clue as completed
3. Display all clues
4. Search clue by keyword
5. Exit
Enter your choice: 4
Enter keyword: L
Found clue: Lap

Scavenger Hunt Menu
1. Add clue
2. Mark next clue as completed
3. Display all clues
4. Search clue by keyword
5. Exit
Enter your choice: 5
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the scavenger hunt program was executed successfully. The clues were stored and retrieved correctly and the output was verified.

Question 4:

A game is played by always eliminating the kth player from the last elimination and played until one player is left. Given N players where each is assigned to a number, find the number of the last remaining player. Write a C program to implement this game using linked list.

Aim:

To develop a C program to simulate the elimination game using circular linked list. The program identifies the last remaining player after repeated eliminations

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Player {
    int id;
    struct Player *next;
} Player;

static Player *create_circle(int n) {
    Player *head = NULL;
    Player *tail = NULL;

    for (int i = 1; i <= n; ++i) {
        Player *node = (Player *) malloc(sizeof(Player));
        if (node == NULL) {
            printf("Memory allocation failed.\n");
            exit(EXIT_FAILURE);
        }
        node->id = i;
        node->next = NULL;

        if (head == NULL) {
            head = tail = node;
        } else {
            tail->next = node;
            tail = node;
        }
    }

    if (tail != NULL) {
        tail->next = head;
    }
    return tail;
}

int main(void) {
    int n;
    int k;

    printf("Enter number of players: ");
    scanf("%d", &n);
    printf("Enter elimination step k: ");
    scanf("%d", &k);
```

```
    if (n <= 0 || k <= 0) {
        printf("N and k must be positive.\n");
        return 0;
    }

    Player *tail = create_circle(n);
    Player *prev = tail;

    printf("Elimination order: ");
    while (prev->next != prev) {
        for (int step = 1; step < k; ++step) {
            prev = prev->next;
        }

        Player *removed = prev->next;
        printf("%d ", removed->id);
        prev->next = removed->next;
        if (removed == tail) {
            tail = prev;
        }
        free(removed);
    }

    printf("\nLast remaining player: %d\n", prev->id);
    free(prev);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Assignment$ cd Ass_2_ds
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./4_josephus_game_linked_list
Enter number of players: 3
Enter elimination step k: 2
Elimination order: 2 1
Last remaining player: 3
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./4_josephus_game_linked_list
Enter number of players: 5
Enter elimination step k: 9
Elimination order: 4 5 3 1
Last remaining player: 2
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./4_josephus_game_linked_list
Enter number of players: 0
Enter elimination step k: 0
N and k must be positive.
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./4_josephus_game_linked_list
Enter number of players: 1
Enter elimination step k: 1
Elimination order:
Last remaining player: 1
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the Josephus game program was executed successfully. The final remaining player was identified correctly and the result was verified.

Question 5:

Write a C program to implement the history list of the file directories, traversal to the previous or next directory using linked list.

Aim:

To write a C program to maintain directory navigation history using linked list. It allows traversal to previous and next directories.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct HistoryNode {
    char path[256];
    struct HistoryNode *prev;
    struct HistoryNode *next;
} HistoryNode;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static HistoryNode *create_node(const char *path) {
    HistoryNode *node = (HistoryNode *) malloc(sizeof(HistoryNode));
    if (node == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(node->path, path, sizeof(node->path) - 1);
    node->path[sizeof(node->path) - 1] = '\0';
    node->prev = NULL;
    node->next = NULL;
    return node;
}

static void free_forward(HistoryNode *node) {
    while (node != NULL) {
        HistoryNode *next = node->next;
        free(node);
        node = next;
    }
}

static void visit_directory(HistoryNode **current, const char *path) {
    HistoryNode *new_node = create_node(path);

    if (*current == NULL) {
        *current = new_node;
        return;
    }
```

```
    free_forward((*current)->next);
    (*current)->next = new_node;
    new_node->prev = *current;
    *current = new_node;
}

static void show_history(HistoryNode *current) {
    if (current == NULL) {
        printf("History is empty.\n");
        return;
    }

    while (current->prev != NULL) {
        current = current->prev;
    }

    printf("\nDirectory history:\n");
    while (current != NULL) {
        printf("%s\n", current->path);
        current = current->next;
    }
}

static void free_all(HistoryNode *current) {
    if (current == NULL) {
        return;
    }

    while (current->prev != NULL) {
        current = current->prev;
    }
    free_forward(current);
}

int main(void) {
    HistoryNode *current = NULL;
    int choice;
    char path[256];

    while (1) {
        printf("\nDirectory History Menu\n");
        printf("1. Visit directory\n");
        printf("2. Go to previous directory\n");
        printf("3. Go to next directory\n");
        printf("4. Show current directory\n");
        printf("5. Show full history\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
```

```
            case 1:
                printf("Enter directory path: ");
                read_line(path, sizeof(path));
                visit_directory(&current, path);
                break;
            case 2:
                if (current != NULL && current->prev != NULL) {
                    current = current->prev;
                    printf("Moved to: %s\n", current->path);
                } else {
                    printf("No previous directory.\n");
                }
                break;
            case 3:
                if (current != NULL && current->next != NULL) {
                    current = current->next;
                    printf("Moved to: %s\n", current->path);
                } else {
                    printf("No next directory.\n");
                }
                break;
            case 4:
                if (current != NULL) {
                    printf("Current directory: %s\n", current->path);
                } else {
                    printf("History is empty.\n");
                }
                break;
            case 5:
                show_history(current);
                break;
            case 6:
                free_all(current);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_all(current);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./5_directory_history_doubly_linked_list

Directory History Menu
1. Visit directory
2. Go to previous directory
3. Go to next directory
4. Show current directory
5. Show full history
6. Exit
Enter your choice: 1
Enter directory path: /home/silicondoc/Documents/Fun Project

Directory History Menu
1. Visit directory
2. Go to previous directory
3. Go to next directory
4. Show current directory
5. Show full history
6. Exit
Enter your choice: 1
Enter directory path: /home/silicondoc/Documents/Fun Project/MiniKernal

Directory History Menu
1. Visit directory
2. Go to previous directory
3. Go to next directory
4. Show current directory
5. Show full history
6. Exit
Enter your choice: 2
Moved to: /home/silicondoc/Documents/Fun Project

Directory History Menu
1. Visit directory
2. Go to previous directory
3. Go to next directory
4. Show current directory
5. Show full history
6. Exit
Enter your choice: 3
Moved to: /home/silicondoc/Documents/Fun Project/MiniKernal
```

```
Directory History Menu
1. Visit directory
2. Go to previous directory
3. Go to next directory
4. Show current directory
5. Show full history
6. Exit
Enter your choice: 4
Current directory: /home/silicondoc/Documents/Fun Project/MiniKernal

Directory History Menu
1. Visit directory
2. Go to previous directory
3. Go to next directory
4. Show current directory
5. Show full history
6. Exit
Enter your choice: 5

Directory history:
/home/silicondoc/Documents/Fun Project
/home/silicondoc/Documents/Fun Project/MiniKernal

Directory History Menu
1. Visit directory
2. Go to previous directory
3. Go to next directory
4. Show current directory
5. Show full history
6. Exit
Enter your choice: 6
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the directory navigation program was executed successfully. The movement between directories was performed correctly and verified.

Question 6:

Write a C program to implementation hash tables using linked list

Aim:

To implement hashing technique using linked list to handle collisions. This program demonstrates separate chaining method.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 13

typedef struct Entry {
    char key[100];
    int value;
    struct Entry *next;
} Entry;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static unsigned int hash_key(const char *key) {
    unsigned int hash = 0;
    while (*key != '\0') {
        hash = hash * 31u + (unsigned char) *key++;
    }
    return hash % TABLE_SIZE;
}

static void insert_entry(Entry *table[], const char *key, int value) {
    unsigned int index = hash_key(key);
    Entry *temp = table[index];

    while (temp != NULL) {
        if (strcmp(temp->key, key) == 0) {
            temp->value = value;
            printf("Updated existing key.\n");
            return;
        }
        temp = temp->next;
    }

    Entry *entry = (Entry *) malloc(sizeof(Entry));
    if (entry == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
```

```
    strncpy(entry->key, key, sizeof(entry->key) - 1);
    entry->key[sizeof(entry->key) - 1] = '\0';
    entry->value = value;
    entry->next = table[index];
    table[index] = entry;
}

static void search_entry(Entry *table[], const char *key) {
    unsigned int index = hash_key(key);
    Entry *temp = table[index];

    while (temp != NULL) {
        if (strcmp(temp->key, key) == 0) {
            printf("Key found. Value = %d\n", temp->value);
            return;
        }
        temp = temp->next;
    }

    printf("Key not found.\n");
}

static void delete_entry(Entry *table[], const char *key) {
    unsigned int index = hash_key(key);
    Entry *temp = table[index];
    Entry *prev = NULL;

    while (temp != NULL && strcmp(temp->key, key) != 0) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Key not found.\n");
        return;
    }

    if (prev == NULL) {
        table[index] = temp->next;
    } else {
        prev->next = temp->next;
    }
    free(temp);
    printf("Key deleted.\n");
}

static void display_table(Entry *table[]) {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        printf("[%d] ", i);
        Entry *temp = table[i];
        while (temp != NULL) {
            printf("(%s, %d) -> ", temp->key, temp->value);
            temp = temp->next;
        }
```

```
        printf("NULL\n");
    }
}

static void free_table(Entry *table[]) {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        Entry *temp = table[i];
        while (temp != NULL) {
            Entry *next = temp->next;
            free(temp);
            temp = next;
        }
    }
}

int main(void) {
    Entry *table[TABLE_SIZE] = {0};
    int choice;
    int value;
    char key[100];

    while (1) {
        printf("\nHash Table Menu\n");
        printf("1. Insert or update key\n");
        printf("2. Search key\n");
        printf("3. Delete key\n");
        printf("4. Display table\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter key: ");
                read_line(key, sizeof(key));
                printf("Enter integer value: ");
                scanf("%d", &value);
                getchar();
                insert_entry(table, key, value);
                break;
            case 2:
                printf("Enter key to search: ");
                read_line(key, sizeof(key));
                search_entry(table, key);
                break;
            case 3:
                printf("Enter key to delete: ");
                read_line(key, sizeof(key));
                delete_entry(table, key);
                break;
            case 4:
                display_table(table);
```

```
            break;
        case 5:
            free_table(table);
            return 0;
        default:
            printf("Invalid choice.\n");
    }
}

free_table(table);
return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./6_hash_table_using_linked_list

Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 1
Enter key: Silicon
Enter integer value: 2

Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 1
Enter key: Doc
Enter integer value: 1

Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 2
Enter key to search: 2
Key not found.
```

```
Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 4
[0] NULL
[1] (Doc, 1) -> NULL
[2] NULL
[3] NULL
[4] NULL
[5] NULL
[6] NULL
[7] NULL
[8] NULL
[9] (Silicon, 2) -> NULL
[10] NULL
[11] NULL
[12] NULL

Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 2
Enter key to search: Doc
Key found. Value = 1

Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 3
Enter key to delete: Doc
Key deleted.
```

```
Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 4
[0] NULL
[1] NULL
[2] NULL
[3] NULL
[4] NULL
[5] NULL
[6] NULL
[7] NULL
[8] NULL
[9] (Silicon, 2) -> NULL
[10] NULL
[11] NULL
[12] NULL

Hash Table Menu
1. Insert or update key
2. Search key
3. Delete key
4. Display table
5. Exit
Enter your choice: 5
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the hash table program was executed successfully. The keys were inserted and displayed correctly and the output was verified.

Question 7:

Write a C program to implement a tree to identify the car manufacturer and car type by providing car name.

Aim:

To develop a C program using tree structure to identify car manufacturer and type. This shows hierarchical data representation.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct CarNode {
    char car_name[100];
    char manufacturer[100];
    char type[100];
    struct CarNode *left;
    struct CarNode *right;
} CarNode;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static CarNode *create_car(const char *name, const char *manufacturer, const char *type) {
    CarNode *node = (CarNode *) malloc(sizeof(CarNode));
    if (node == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(node->car_name, name, sizeof(node->car_name) - 1);
    node->car_name[sizeof(node->car_name) - 1] = '\0';
    strncpy(node->manufacturer, manufacturer, sizeof(node->manufacturer) - 1);
    node->manufacturer[sizeof(node->manufacturer) - 1] = '\0';
    strncpy(node->type, type, sizeof(node->type) - 1);
    node->type[sizeof(node->type) - 1] = '\0';
    node->left = NULL;
    node->right = NULL;
    return node;
}

static CarNode *insert_car(CarNode *root, const char *name, const char *manufacturer, const char *type) {
    if (root == NULL) {
        return create_car(name, manufacturer, type);
    }

    int compare = strcmp(name, root->car_name);
    if (compare < 0) {
        root->left = insert_car(root->left, name, manufacturer, type);
```

```
    } else if (compare > 0) {
        root->right = insert_car(root->right, name, manufacturer, type);
    } else {
        strncpy(root->manufacturer, manufacturer, sizeof(root->manufacturer) - 1);
        root->manufacturer[sizeof(root->manufacturer) - 1] = '\0';
        strncpy(root->type, type, sizeof(root->type) - 1);
        root->type[sizeof(root->type) - 1] = '\0';
    }
    return root;
}

static CarNode *search_car(CarNode *root, const char *name) {
    while (root != NULL) {
        int compare = strcmp(name, root->car_name);
        if (compare == 0) {
            return root;
        }
        root = (compare < 0) ? root->left : root->right;
    }
    return NULL;
}

static void inorder(CarNode *root) {
    if (root == NULL) {
        return;
    }
    inorder(root->left);
    printf("Car: %-20s Manufacturer: %-15s Type: %s\n", root->car_name, root->manufacturer, root->type);
    inorder(root->right);
}

static void free_tree(CarNode *root) {
    if (root == NULL) {
        return;
    }
    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

int main(void) {
    CarNode *root = NULL;
    int choice;
    char name[100];
    char manufacturer[100];
    char type[100];

    while (1) {
        printf("\nCar Lookup Tree Menu\n");
        printf("1. Add or update car\n");
        printf("2. Search by car name\n");
        printf("3. Display all cars\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
```

```
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter car name: ");
                read_line(name, sizeof(name));
                printf("Enter manufacturer: ");
                read_line(manufacturer, sizeof(manufacturer));
                printf("Enter car type: ");
                read_line(type, sizeof(type));
                root = insert_car(root, name, manufacturer, type);
                break;
            case 2:
                printf("Enter car name to search: ");
                read_line(name, sizeof(name));
                {
                    CarNode *result = search_car(root, name);
                    if (result != NULL) {
                        printf("Manufacturer: %s\nType: %s\n", result->manufacturer, result->type);
                    } else {
                        printf("Car not found.\n");
                    }
                }
                break;
            case 3:
                if (root == NULL) {
                    printf("Tree is empty.\n");
                } else {
                    inorder(root);
                }
                break;
            case 4:
                free_tree(root);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_tree(root);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./7_car_lookup_tree

Car Lookup Tree Menu
1. Add or update car
2. Search by car name
3. Display all cars
4. Exit
Enter your choice: 1
Enter car name: Sierra
Enter manufacturer: Tata
Enter car type: mid size suv

Car Lookup Tree Menu
1. Add or update car
2. Search by car name
3. Display all cars
4. Exit
Enter your choice: 1
Enter car name: Thar
Enter manufacturer: Mahindra
Enter car type: off road SUV

Car Lookup Tree Menu
1. Add or update car
2. Search by car name
3. Display all cars
4. Exit
Enter your choice: 2
Enter car name to search: Thar
Manufacturer: Mahindra
Type: off road SUV
```

```
Car Lookup Tree Menu
1. Add or update car
2. Search by car name
3. Display all cars
4. Exit
Enter your choice: 3
Car: Sierra                Manufacturer: Tata            Type: mid size suv
Car: Thar                  Manufacturer: Mahindra        Type: off road SUV

Car Lookup Tree Menu
1. Add or update car
2. Search by car name
3. Display all cars
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the car identification program was executed successfully. The required car details were displayed correctly and verified.

Question 8:

Write a C program to implement a Graph using linked list.

Aim:

To represent and implement a graph using adjacency list representation. This program demonstrates graph traversal and storage.

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 20

typedef struct AdjNode {
    int dest;
    struct AdjNode *next;
} AdjNode;

typedef struct {
    AdjNode *head[MAX_VERTICES];
    int vertices;
} Graph;

static AdjNode *create_node(int dest) {
    AdjNode *node = (AdjNode *) malloc(sizeof(AdjNode));
    if (node == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    node->dest = dest;
    node->next = NULL;
    return node;
}

static void init_graph(Graph *graph, int vertices) {
    graph->vertices = vertices;
    for (int i = 0; i < MAX_VERTICES; ++i) {
        graph->head[i] = NULL;
    }
}

static void add_edge(Graph *graph, int src, int dest) {
    if (src < 0 || src >= graph->vertices || dest < 0 || dest >= graph->vertices) {
        printf("Invalid vertices.\n");
        return;
    }

    AdjNode *node = create_node(dest);
    node->next = graph->head[src];
    graph->head[src] = node;

    node = create_node(src);
    node->next = graph->head[dest];
    graph->head[dest] = node;
```

```
}

static void display_graph(const Graph *graph) {
    for (int i = 0; i < graph->vertices; ++i) {
        printf("%d:", i);
        AdjNode *temp = graph->head[i];
        while (temp != NULL) {
            printf(" %d", temp->dest);
            temp = temp->next;
        }
        printf("\n");
    }
}

static void bfs(const Graph *graph, int start) {
    if (start < 0 || start >= graph->vertices) {
        printf("Invalid start vertex.\n");
        return;
    }

    int visited[MAX_VERTICES] = {0};
    int queue[MAX_VERTICES];
    int front = 0;
    int rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    printf("BFS traversal: ");
    while (front < rear) {
        int vertex = queue[front++];
        printf("%d ", vertex);

        AdjNode *temp = graph->head[vertex];
        while (temp != NULL) {
            if (!visited[temp->dest]) {
                visited[temp->dest] = 1;
                queue[rear++] = temp->dest;
            }
            temp = temp->next;
        }
    }
    printf("\n");
}

static void dfs_recursive(const Graph *graph, int vertex, int visited[]) {
    visited[vertex] = 1;
    printf("%d ", vertex);

    AdjNode *temp = graph->head[vertex];
    while (temp != NULL) {
        if (!visited[temp->dest]) {
            dfs_recursive(graph, temp->dest, visited);
        }
        temp = temp->next;
```

```
    }
}

static void dfs(const Graph *graph, int start) {
    if (start < 0 || start >= graph->vertices) {
        printf("Invalid start vertex.\n");
        return;
    }

    int visited[MAX_VERTICES] = {0};
    printf("DFS traversal: ");
    dfs_recursive(graph, start, visited);
    printf("\n");
}

static void free_graph(Graph *graph) {
    for (int i = 0; i < graph->vertices; ++i) {
        AdjNode *temp = graph->head[i];
        while (temp != NULL) {
            AdjNode *next = temp->next;
            free(temp);
            temp = next;
        }
    }
}

int main(void) {
    Graph graph;
    int vertices;
    int choice;
    int src;
    int dest;
    int start;

    printf("Enter number of vertices (max %d): ", MAX_VERTICES);
    scanf("%d", &vertices);
    if (vertices <= 0 || vertices > MAX_VERTICES) {
        printf("Invalid number of vertices.\n");
        return 0;
    }

    init_graph(&graph, vertices);

    while (1) {
        printf("\nGraph Menu\n");
        printf("1. Add edge\n");
        printf("2. Display adjacency list\n");
        printf("3. BFS traversal\n");
        printf("4. DFS traversal\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
```

```
        switch (choice) {
            case 1:
                printf("Enter source and destination: ");
                scanf("%d %d", &src, &dest);
                add_edge(&graph, src, dest);
                break;
            case 2:
                display_graph(&graph);
                break;
            case 3:
                printf("Enter start vertex: ");
                scanf("%d", &start);
                bfs(&graph, start);
                break;
            case 4:
                printf("Enter start vertex: ");
                scanf("%d", &start);
                dfs(&graph, start);
                break;
            case 5:
                free_graph(&graph);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_graph(&graph);
    return 0;
}
```

Output:

```
Graph Menu
1. Add edge
2. Display adjacency list
3. BFS traversal
4. DFS traversal
5. Exit
Enter your choice: 4
Enter start vertex: 0
DFS traversal: 0 2 1 3

Graph Menu
1. Add edge
2. Display adjacency list
3. BFS traversal
4. DFS traversal
5. Exit
Enter your choice: 5
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./8_graph_using_linked_list
Enter number of vertices (max 20): 4

Graph Menu
1. Add edge
2. Display adjacency list
3. BFS traversal
4. DFS traversal
5. Exit
Enter your choice: 1
Enter source and destination: 0 1

Graph Menu
1. Add edge
2. Display adjacency list
3. BFS traversal
4. DFS traversal
5. Exit
Enter your choice: 1
Enter source and destination: 0 2

Graph Menu
1. Add edge
2. Display adjacency list
3. BFS traversal
4. DFS traversal
5. Exit
Enter your choice: 1
Enter source and destination: 1 3

Graph Menu
1. Add edge
2. Display adjacency list
3. BFS traversal
4. DFS traversal
5. Exit
Enter your choice: 2
0: 2 1
1: 3 0
2: 0
3: 1

Graph Menu
1. Add edge
2. Display adjacency list
3. BFS traversal
4. DFS traversal
5. Exit
Enter your choice: 3
Enter start vertex: 0
BFS traversal: 0 2 1 3
```

Result:

Thus the graph implementation program was executed successfully. The graph structure was created and displayed correctly.

Question 9:

Write a C program to implement a Scheduler which uses linked list

Aim:

To write a C program to simulate task scheduling using linked list. Tasks are arranged and executed sequentially.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Task {
    char name[100];
    int arrival_time;
    int burst_time;
    struct Task *next;
} Task;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Task *create_task(const char *name, int arrival, int burst) {
    Task *task = (Task *) malloc(sizeof(Task));
    if (task == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(task->name, name, sizeof(task->name) - 1);
    task->name[sizeof(task->name) - 1] = '\0';
    task->arrival_time = arrival;
    task->burst_time = burst;
    task->next = NULL;
    return task;
}

static void schedule_task(Task **head, const char *name, int arrival, int burst) {
    Task *task = create_task(name, arrival, burst);

    if (*head == NULL || arrival < (*head)->arrival_time) {
        task->next = *head;
        *head = task;
        return;
    }

    Task *temp = *head;
    while (temp->next != NULL && temp->next->arrival_time <= arrival) {
        temp = temp->next;
    }
    task->next = temp->next;
```

```
    temp->next = task;
}

static void run_next_task(Task **head, int *current_time) {
    if (*head == NULL) {
        printf("No tasks scheduled.\n");
        return;
    }

    Task *task = *head;
    *head = task->next;

    if (*current_time < task->arrival_time) {
        *current_time = task->arrival_time;
    }

    printf("Running task %s from time %d to %d\n",
           task->name, *current_time, *current_time + task->burst_time);
    *current_time += task->burst_time;
    free(task);
}

static void display_tasks(Task *head) {
    if (head == NULL) {
        printf("Scheduler queue is empty.\n");
        return;
    }

    printf("\nPending tasks:\n");
    while (head != NULL) {
        printf("Task: %-15s Arrival: %-4d Burst: %d\n",
               head->name, head->arrival_time, head->burst_time);
        head = head->next;
    }
}

static void free_tasks(Task *head) {
    while (head != NULL) {
        Task *next = head->next;
        free(head);
        head = next;
    }
}

int main(void) {
    Task *head = NULL;
    int current_time = 0;
    int choice;
    int arrival;
    int burst;
    char name[100];

    while (1) {
        printf("\nScheduler Menu\n");
        printf("1. Add task\n");
```

```
        printf("2. Run next task\n");
        printf("3. Display pending tasks\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter task name: ");
                read_line(name, sizeof(name));
                printf("Enter arrival time: ");
                scanf("%d", &arrival);
                printf("Enter burst time: ");
                scanf("%d", &burst);
                getchar();
                schedule_task(&head, name, arrival, burst);
                break;
            case 2:
                run_next_task(&head, &current_time);
                break;
            case 3:
                display_tasks(head);
                break;
            case 4:
                free_tasks(head);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_tasks(head);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./9_scheduler_linked_list

Scheduler Menu
1. Add task
2. Run next task
3. Display pending tasks
4. Exit
Enter your choice: 1
Enter task name: SetupEnvironment
Enter arrival time: 0
Enter burst time: 10

Scheduler Menu
1. Add task
2. Run next task
3. Display pending tasks
4. Exit
Enter your choice: 1
Enter task name: CompileCode
Enter arrival time: 5
Enter burst time: 15

Scheduler Menu
1. Add task
2. Run next task
3. Display pending tasks
4. Exit
Enter your choice: 3

Pending tasks:
Task: SetupEnvironment Arrival: 0 Burst: 10
Task: CompileCode Arrival: 5 Burst: 15

Scheduler Menu
1. Add task
2. Run next task
3. Display pending tasks
4. Exit
Enter your choice: 2
Running task SetupEnvironment from time 0 to 10

Scheduler Menu
1. Add task
2. Run next task
3. Display pending tasks
4. Exit
Enter your choice: 2
Running task CompileCode from time 10 to 25

Scheduler Menu
1. Add task
2. Run next task
3. Display pending tasks
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the scheduler program was executed successfully. The tasks were scheduled in proper order and verified.

Question 10:

Write a C program to implement device node adding and deletion using linked list

Aim:

To develop a C program to add and delete device nodes using linked list. This helps in dynamic device management.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Device {
    int id;
    char name[100];
    char type[100];
    struct Device *next;
} Device;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Device *create_device(int id, const char *name, const char *type) {
    Device *device = (Device *) malloc(sizeof(Device));
    if (device == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    device->id = id;
    strncpy(device->name, name, sizeof(device->name) - 1);
    device->name[sizeof(device->name) - 1] = '\0';
    strncpy(device->type, type, sizeof(device->type) - 1);
    device->type[sizeof(device->type) - 1] = '\0';
    device->next = NULL;
    return device;
}

static void add_device(Device **head, int id, const char *name, const char *type) {
    Device *device = create_device(id, name, type);
    if (*head == NULL) {
        *head = device;
        return;
    }

    Device *temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = device;
```

```
}

static void delete_device(Device **head, int id) {
    Device *temp = *head;
    Device *prev = NULL;

    while (temp != NULL && temp->id != id) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Device not found.\n");
        return;
    }

    if (prev == NULL) {
        *head = temp->next;
    } else {
        prev->next = temp->next;
    }
    free(temp);
    printf("Device deleted successfully.\n");
}

static void display_devices(Device *head) {
    if (head == NULL) {
        printf("No devices available.\n");
        return;
    }

    printf("\nDevice list:\n");
    while (head != NULL) {
        printf("ID: %-5d Name: %-15s Type: %s\n", head->id, head->name, head->type);
        head = head->next;
    }
}

static void free_devices(Device *head) {
    while (head != NULL) {
        Device *next = head->next;
        free(head);
        head = next;
    }
}

int main(void) {
    Device *head = NULL;
    int choice;
    int id;
    char name[100];
    char type[100];

    while (1) {
        printf("\nDevice Node Menu\n");
```

```
        printf("1. Add device node\n");
        printf("2. Delete device node\n");
        printf("3. Display devices\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter device ID: ");
                scanf("%d", &id);
                getchar();
                printf("Enter device name: ");
                read_line(name, sizeof(name));
                printf("Enter device type: ");
                read_line(type, sizeof(type));
                add_device(&head, id, name, type);
                break;
            case 2:
                printf("Enter device ID to delete: ");
                scanf("%d", &id);
                getchar();
                delete_device(&head, id);
                break;
            case 3:
                display_devices(head);
                break;
            case 4:
                free_devices(head);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_devices(head);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./10_device_node_management_li

Device Node Menu
1. Add device node
2. Delete device node
3. Display devices
4. Exit
Enter your choice: 1
Enter device ID: 101
Enter device name: AudioInterface
Enter device type: Output Device

Device Node Menu
1. Add device node
2. Delete device node
3. Display devices
4. Exit
Enter your choice: 1
Enter device ID: 102
Enter device name: Trackpad
Enter device type: Input Device

Device Node Menu
1. Add device node
2. Delete device node
3. Display devices
4. Exit
Enter your choice: 3

Device list:
ID: 101 Name: AudioInterface Type: Output Device
ID: 102 Name: Trackpad Type: Input Device

Device Node Menu
1. Add device node
2. Delete device node
3. Display devices
4. Exit
Enter your choice: 2
Enter device ID to delete: 101
Device deleted successfully.

Device Node Menu
1. Add device node
2. Delete device node
3. Display devices
4. Exit
Enter your choice: 3

Device list:
ID: 102 Name: Trackpad Type: Input Device

Device Node Menu
1. Add device node
2. Delete device node
3. Display devices
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the device node management program was executed successfully. The device list operations were performed correctly.

Question 11:

Write a C program to implement string reversal using stack.

Aim:

To write a C program to reverse a given string using stack data structure. This demonstrates LIFO principle.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_SIZE 500

typedef struct {
    char items[MAX_SIZE];
    int top;
} Stack;

static void push(Stack *stack, char value) {
    if (stack->top < MAX_SIZE - 1) {
        stack->items[++stack->top] = value;
    }
}

static char pop(Stack *stack) {
    if (stack->top >= 0) {
        return stack->items[stack->top--];
    }
    return '\0';
}

int main(void) {
    char input[MAX_SIZE];
    Stack stack = {{0}, -1};

    printf("Enter a string: ");
    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }
    input[strcspn(input, "\n")] = '\0';

    for (size_t i = 0; input[i] != '\0'; ++i) {
        push(&stack, input[i]);
    }

    printf("Reversed string: ");
    while (stack.top >= 0) {
        putchar(pop(&stack));
    }
    printf("\n");

    return 0;
```

}

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./11_string_reversal_using_sta
Enter a string: Data Structures
Reversed string: serutcurtS ataD

(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./11_string_reversal_using_sta
Enter a string: silicondoc
Reversed string: codnocilis
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the string reversal program was executed successfully. The reversed string was obtained correctly and verified.

Question 12:

Write a C program to solve maze problem with back trace using stack.

Aim:

To implement maze solving using stack and backtracking technique. This program finds a valid path from source to destination.

Code:

```
#include <stdio.h>

#define MAX 20

typedef struct {
    int row;
    int col;
    int dir;
} Cell;

static int is_valid(int rows, int cols, int row, int col) {
    return row >= 0 && row < rows && col >= 0 && col < cols;
}

int main(void) {
    int rows;
    int cols;
    int maze[MAX][MAX];
    int visited[MAX][MAX] = {{0}};
    Cell stack[MAX * MAX];
    int top = -1;
    int sr;
    int sc;
    int er;
    int ec;
    int found = 0;
    const int dr[4] = {-1, 0, 1, 0};
    const int dc[4] = {0, 1, 0, -1};

    printf("Enter number of rows and columns (max %d): ", MAX);
    scanf("%d %d", &rows, &cols);
    if (rows <= 0 || cols <= 0 || rows > MAX || cols > MAX) {
        printf("Invalid maze size.\n");
        return 0;
    }

    printf("Enter maze matrix (0 for open path, 1 for wall):\n");
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            scanf("%d", &maze[i][j]);
        }
    }

    printf("Enter start cell (row col): ");
    scanf("%d %d", &sr, &sc);
    printf("Enter end cell (row col): ");
```

```
    scanf("%d %d", &er, &ec);

    if (!is_valid(rows, cols, sr, sc) || !is_valid(rows, cols, er, ec) ||
        maze[sr][sc] == 1 || maze[er][ec] == 1) {
        printf("Invalid start or end cell.\n");
        return 0;
    }

    stack[++top] = (Cell){sr, sc, 0};
    visited[sr][sc] = 1;

    while (top >= 0) {
        Cell *current = &stack[top];

        if (current->row == er && current->col == ec) {
            found = 1;
            break;
        }

        int moved = 0;
        while (current->dir < 4) {
            int direction = current->dir++;
            int nr = current->row + dr[direction];
            int nc = current->col + dc[direction];

            if (is_valid(rows, cols, nr, nc) && maze[nr][nc] == 0 && !visited[nr][nc]) {
                visited[nr][nc] = 1;
                stack[++top] = (Cell){nr, nc, 0};
                moved = 1;
                break;
            }
        }

        if (!moved) {
            --top;
        }
    }

    if (!found) {
        printf("No path found through the maze.\n");
        return 0;
    }

    printf("Path found using stack backtracking:\n");
    for (int i = 0; i <= top; ++i) {
        printf("(%d,%d)", stack[i].row, stack[i].col);
        if (i < top) {
            printf(" -> ");
        }
    }
    printf("\n");

    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./12_maze_backtrace_using_stac
Enter number of rows and columns (max 20): 4 4
Enter maze matrix (0 for open path, 1 for wall):
0 0 1 1
1 0 0 1
1 1 0 0
1 1 1 0
Enter start cell (row col): 0 0
Enter end cell (row col): 3 3
Path found using stack backtracking:
(0,0) -> (0,1) -> (1,1) -> (1,2) -> (2,2) -> (2,3) -> (3,3)
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the maze solving program was executed successfully. The correct path was identified and the output was verified.

Question 13:

Write a C program to implement and check balanced parentheses using stack.

Aim:

To develop a C program to check balanced parentheses using stack. This helps to validate expressions.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_EXPR 500

typedef struct {
    char items[MAX_EXPR];
    int top;
} Stack;

static void push(Stack *stack, char value) {
    if (stack->top < MAX_EXPR - 1) {
        stack->items[++stack->top] = value;
    }
}

static char pop(Stack *stack) {
    if (stack->top >= 0) {
        return stack->items[stack->top--];
    }
    return '\0';
}

static int is_matching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

int main(void) {
    char expression[MAX_EXPR];
    Stack stack = {{0}, -1};
    int balanced = 1;

    printf("Enter an expression: ");
    if (fgets(expression, sizeof(expression), stdin) == NULL) {
        return 0;
    }

    for (size_t i = 0; expression[i] != '\0'; ++i) {
        char ch = expression[i];
        if (ch == '(' || ch == '[' || ch == '{') {
            push(&stack, ch);
        } else if (ch == ')' || ch == ']' || ch == '}') {
            if (stack.top == -1 || !is_matching(pop(&stack), ch)) {
                balanced = 0;
```

```
            break;
        }
    }
}

if (stack.top != -1) {
    balanced = 0;
}

if (balanced) {
    printf("The parentheses are balanced.\n");
} else {
    printf("The parentheses are not balanced.\n");
}

return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./13_balanced_parentheses_usin
Enter an expression: {[()]}
The parentheses are balanced.

(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./13_balanced_parentheses_usin
Enter an expression: {[(])}
The parentheses are not balanced.

(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./13_balanced_parentheses_usin
Enter an expression: ((()))
The parentheses are balanced.
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the parentheses checking program was executed successfully. The expression validity was determined correctly.

Question 14:

Write a C program to implement the memory management model using stack.

Aim:

To simulate memory allocation and deallocation operations using stack. This program shows stack based memory model.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_BLOCKS 100
#define TOTAL_MEMORY 1024

typedef struct {
    char process_name[100];
    int start_address;
    int size;
} MemoryBlock;

typedef struct {
    MemoryBlock blocks[MAX_BLOCKS];
    int top;
    int used_memory;
} MemoryStack;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static void allocate_memory(MemoryStack *stack, const char *name, int size) {
    if (stack->top >= MAX_BLOCKS - 1) {
        printf("Memory stack is full.\n");
        return;
    }

    if (size <= 0 || stack->used_memory + size > TOTAL_MEMORY) {
        printf("Allocation failed. Not enough memory.\n");
        return;
    }

    ++stack->top;
    strncpy(stack->blocks[stack->top].process_name, name,
            sizeof(stack->blocks[stack->top].process_name) - 1);
    stack->blocks[stack->top].process_name[sizeof(stack->blocks[stack->top].process_name) - 1] = '\0';
    stack->blocks[stack->top].start_address = stack->used_memory;
    stack->blocks[stack->top].size = size;
    stack->used_memory += size;

    printf("Allocated %d units to %s at address %d\n",
```

```
        size,
        stack->blocks[stack->top].process_name,
        stack->blocks[stack->top].start_address);
}

static void deallocate_memory(MemoryStack *stack) {
    if (stack->top < 0) {
        printf("No memory block to deallocate.\n");
        return;
    }

    MemoryBlock block = stack->blocks[stack->top--];
    stack->used_memory -= block.size;
    printf("Deallocated %s of size %d\n", block.process_name, block.size);
}

static void display_memory(const MemoryStack *stack) {
    if (stack->top < 0) {
        printf("No memory allocations present.\n");
        return;
    }

    printf("\nAllocated memory blocks:\n");
    for (int i = stack->top; i >= 0; --i) {
        printf("Process: %-15s Start: %-4d Size: %d\n",
               stack->blocks[i].process_name,
               stack->blocks[i].start_address,
               stack->blocks[i].size);
    }
    printf("Used memory: %d / %d\n", stack->used_memory, TOTAL_MEMORY);
}

int main(void) {
    MemoryStack stack = {.top = -1, .used_memory = 0};
    int choice;
    int size;
    char name[100];

    while (1) {
        printf("\nMemory Management Menu\n");
        printf("1. Allocate memory\n");
        printf("2. Deallocate memory\n");
        printf("3. Display memory stack\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter process name: ");
                read_line(name, sizeof(name));
                printf("Enter memory size to allocate: ");
```

```
            scanf("%d", &size);
            getchar();
            allocate_memory(&stack, name, size);
            break;
        case 2:
            deallocate_memory(&stack);
            break;
        case 3:
            display_memory(&stack);
            break;
        case 4:
            return 0;
        default:
            printf("Invalid choice.\n");
    }
  }

  return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./14_memory_management_using_s

Memory Management Menu
1. Allocate memory
2. Deallocate memory
3. Display memory stack
4. Exit
Enter your choice: 1
Enter process name: GoogleChrome
Enter memory size to allocate: 300
Allocated 300 units to GoogleChrome at address 0

Memory Management Menu
1. Allocate memory
2. Deallocate memory
3. Display memory stack
4. Exit
Enter your choice: 1
Enter process name: VSCode
Enter memory size to allocate: 250
Allocated 250 units to VSCode at address 300

Memory Management Menu
1. Allocate memory
2. Deallocate memory
3. Display memory stack
4. Exit
Enter your choice: 3

Allocated memory blocks:
Process: VSCode Start: 300 Size: 250
Process: GoogleChrome Start: 0 Size: 300
Used memory: 550 / 1024

Memory Management Menu
1. Allocate memory
2. Deallocate memory
3. Display memory stack
4. Exit
Enter your choice: 2
Deallocated VSCode of size 250

Memory Management Menu
1. Allocate memory
2. Deallocate memory
3. Display memory stack
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the memory management program was executed successfully. The stack operations were performed correctly.

Question 15:

Write a C program to implement the undo operation using stack

Aim:

To implement undo functionality using stack data structure. This program restores previous states.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TEXT 1000

typedef struct StateNode {
    char state[MAX_TEXT];
    struct StateNode *next;
} StateNode;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static void push_state(StateNode **top, const char *text) {
    StateNode *node = (StateNode *) malloc(sizeof(StateNode));
    if (node == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(node->state, text, sizeof(node->state) - 1);
    node->state[sizeof(node->state) - 1] = '\0';
    node->next = *top;
    *top = node;
}

static int pop_state(StateNode **top, char *text) {
    if (*top == NULL) {
        return 0;
    }

    StateNode *node = *top;
    *top = node->next;
    strncpy(text, node->state, MAX_TEXT - 1);
    text[MAX_TEXT - 1] = '\0';
    free(node);
    return 1;
}

static void free_states(StateNode *top) {
    while (top != NULL) {
        StateNode *next = top->next;
```

```
        free(top);
        top = next;
    }
}

int main(void) {
    StateNode *undo_stack = NULL;
    char document[MAX_TEXT] = "";
    char text[MAX_TEXT];
    int choice;
    int count;

    while (1) {
        printf("\nUndo Stack Menu\n");
        printf("1. Append text\n");
        printf("2. Delete last N characters\n");
        printf("3. Undo last operation\n");
        printf("4. Display document\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                push_state(&undo_stack, document);
                printf("Enter text to append: ");
                read_line(text, sizeof(text));
                if (strlen(document) + strlen(text) < MAX_TEXT) {
                    strcat(document, text);
                } else {
                    printf("Append skipped: document would exceed limit.\n");
                }
                break;
            case 2:
                push_state(&undo_stack, document);
                printf("Enter number of characters to delete: ");
                scanf("%d", &count);
                getchar();
                if (count < 0) {
                    count = 0;
                }
                if ((size_t) count > strlen(document)) {
                    count = (int) strlen(document);
                }
                document[strlen(document) - (size_t) count] = '\0';
                break;
            case 3:
                if (!pop_state(&undo_stack, document)) {
                    printf("Nothing to undo.\n");
                }
                break;
            case 4:
```

```
            printf("Document: %s\n", document);
            break;
        case 5:
            free_states(undo_stack);
            return 0;
        default:
            printf("Invalid choice.\n");
    }}
    free_states(undo_stack);
    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./15_undo_operation_using_stac

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
5. Exit
Enter your choice: 1
Enter text to append: Silicon

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
5. Exit
Enter your choice: 1
Enter text to append: Doc

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
5. Exit
Enter your choice: 4
Document: Silicon Doc

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
5. Exit
Enter your choice: 2
Enter number of characters to delete: 4

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
5. Exit
Enter your choice: 4
Document: Silicon

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
```

```
5. Exit
Enter your choice: 3

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
5. Exit
Enter your choice: 4
Document: Silicon Doc

Undo Stack Menu
1. Append text
2. Delete last N characters
3. Undo last operation
4. Display document
5. Exit
Enter your choice: 5
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the undo operation program was executed successfully. The previous data was restored correctly.

Question 16:

Write a C program to implement the stack to store and restore data using function calls.

Aim:

To simulate function call handling using stack. This helps to understand call and return mechanism

Code:

```
#include <stdio.h>

#define MAX_DEPTH 100

typedef struct {
    char function_name[50];
    int argument;
} CallFrame;

typedef struct {
    CallFrame frames[MAX_DEPTH];
    int top;
} CallStack;

static void push(CallStack *stack, const char *name, int argument) {
    if (stack->top >= MAX_DEPTH - 1) {
        return;
    }
    ++stack->top;
    snprintf(stack->frames[stack->top].function_name,
            sizeof(stack->frames[stack->top].function_name),
            "%s",
            name);
    stack->frames[stack->top].argument = argument;
}

static void pop(CallStack *stack) {
    if (stack->top >= 0) {
        --stack->top;
    }
}

static void display_stack(const CallStack *stack) {
    printf("Current call stack:\n");
    for (int i = stack->top; i >= 0; --i) {
        printf("  %s(%d)\n", stack->frames[i].function_name, stack->frames[i].argument);
    }
}

static int sum_to_n(int n, CallStack *stack) {
    push(stack, "sum_to_n", n);
    printf("\nStored data while entering function call:\n");
    display_stack(stack);

    int result;
    if (n == 0) {
        result = 0;
    } else {
```

```
        result = n + sum_to_n(n - 1, stack);
    }

    printf("Restoring data while returning from %s(%d)\n",
           stack->frames[stack->top].function_name,
           stack->frames[stack->top].argument);
    pop(stack);
    return result;
}

int main(void) {
    int n;
    CallStack stack = {.top = -1};

    printf("Enter n to calculate sum from 1 to n: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("n must be non-negative.\n");
        return 0;   }
    printf("Result: %d\n", sum_to_n(n, &stack));
    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./16_function_call_stack_stor
Enter n to calculate sum from 1 to n: 3

Stored data while entering function call:
Current call stack:
sum_to_n(3)

Stored data while entering function call:
Current call stack:
sum_to_n(2)
sum_to_n(3)

Stored data while entering function call:
Current call stack:
sum_to_n(1)
sum_to_n(2)
sum_to_n(3)

Stored data while entering function call:
Current call stack:
sum_to_n(0)
sum_to_n(1)
sum_to_n(2)
sum_to_n(3)
Restoring data while returning from sum_to_n(0)
Restoring data while returning from sum_to_n(1)
Restoring data while returning from sum_to_n(2)
Restoring data while returning from sum_to_n(3)
Result: 6
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the function call stack program was executed successfully. The data was stored and restored correctly.

Question 17:

Write a C program to implement token number announcement system model for Banks and Hospitals using Queue

Aim:

To develop a C program to simulate token system using queue. This program follows FIFO principle.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct TokenNode {
    int token_number;
    char counter_name[100];
    struct TokenNode *next;
} TokenNode;

typedef struct {
    TokenNode *front;
    TokenNode *rear;
    int next_token;
} TokenQueue;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static void enqueue(TokenQueue *queue, const char *counter_name) {
    TokenNode *node = (TokenNode *) malloc(sizeof(TokenNode));
    if (node == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    node->token_number = queue->next_token++;
    strncpy(node->counter_name, counter_name, sizeof(node->counter_name) - 1);
    node->counter_name[sizeof(node->counter_name) - 1] = '\0';
    node->next = NULL;

    if (queue->rear == NULL) {
        queue->front = queue->rear = node;
    } else {
        queue->rear->next = node;
        queue->rear = node;
    }

    printf("Issued token %d for %s\n", node->token_number, node->counter_name);
}

static void dequeue(TokenQueue *queue) {
```

```
    if (queue->front == NULL) {
        printf("No tokens waiting.\n");
        return;
    }

    TokenNode *temp = queue->front;
    printf("Now serving token %d at %s\n", temp->token_number, temp->counter_name);
    queue->front = temp->next;
    if (queue->front == NULL) {
        queue->rear = NULL;
    }
    free(temp);
}

static void display_queue(const TokenQueue *queue) {
    if (queue->front == NULL) {
        printf("No tokens in queue.\n");
        return;
    }

    printf("\nWaiting tokens:\n");
    TokenNode *temp = queue->front;
    while (temp != NULL) {
        printf("Token %d -> %s\n", temp->token_number, temp->counter_name);
        temp = temp->next;
    }
}

static void free_queue(TokenQueue *queue) {
    while (queue->front != NULL) {
        TokenNode *next = queue->front->next;
        free(queue->front);
        queue->front = next;
    }
    queue->rear = NULL;
}

int main(void) {
    TokenQueue queue = {NULL, NULL, 1};
    int choice;
    char counter_name[100];

    while (1) {
        printf("\nToken Announcement Queue Menu\n");
        printf("1. Generate token\n");
        printf("2. Announce next token\n");
        printf("3. Display waiting queue\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
```

```
            case 1:
                printf("Enter service counter name (Bank/Hospital desk): ");
                read_line(counter_name, sizeof(counter_name));
                enqueue(&queue, counter_name);
                break;
            case 2:
                dequeue(&queue);
                break;
            case 3:
                display_queue(&queue);
                break;
            case 4:
                free_queue(&queue);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free_queue(&queue);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./17_token_announcement_queue

Token Announcement Queue Menu
1. Generate token
2. Announce next token
3. Display waiting queue
4. Exit
Enter your choice: 1
Enter service counter name (Bank/Hospital desk): Cashier 1
Issued token 1 for Cashier 1

Token Announcement Queue Menu
1. Generate token
2. Announce next token
3. Display waiting queue
4. Exit
Enter your choice: 1
Enter service counter name (Bank/Hospital desk): Information Desk
Issued token 2 for Information Desk

Token Announcement Queue Menu
1. Generate token
2. Announce next token
3. Display waiting queue
4. Exit
Enter your choice: 3

Waiting tokens:
Token 1 -> Cashier 1
Token 2 -> Information Desk

Token Announcement Queue Menu
1. Generate token
2. Announce next token
3. Display waiting queue
4. Exit
Enter your choice: 2
Now serving token 1 at Cashier 1

Token Announcement Queue Menu
1. Generate token
2. Announce next token
3. Display waiting queue
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the token system program was executed successfully. The tokens were processed in correct order.

Question 18:

Write a C program to implement reservation or ticket counter model using Queue.

Aim:

To implement ticket reservation model using queue data structure. Customers are served sequentially.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Customer {
    char name[100];
    int tickets;
    struct Customer *next;
} Customer;

typedef struct {
    Customer *front;
    Customer *rear;
} Queue;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static void enqueue(Queue *queue, const char *name, int tickets) {
    Customer *customer = (Customer *) malloc(sizeof(Customer));
    if (customer == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    strncpy(customer->name, name, sizeof(customer->name) - 1);
    customer->name[sizeof(customer->name) - 1] = '\0';
    customer->tickets = tickets;
    customer->next = NULL;

    if (queue->rear == NULL) {
        queue->front = queue->rear = customer;
    } else {
        queue->rear->next = customer;
        queue->rear = customer;
    }
}

static void serve_customer(Queue *queue) {
    if (queue->front == NULL) {
```

```
        printf("No reservation requests pending.\n");
        return;
    }

    Customer *customer = queue->front;
    printf("Booking %d ticket(s) for %s\n", customer->tickets, customer->name);
    queue->front = customer->next;
    if (queue->front == NULL) {
        queue->rear = NULL;
    }
    free(customer);
}

static void display_queue(const Queue *queue) {
    if (queue->front == NULL) {
        printf("Ticket queue is empty.\n");
        return;
    }

    Customer *temp = queue->front;
    while (temp != NULL) {
        printf("Customer: %-15s Tickets: %d\n", temp->name, temp->tickets);
        temp = temp->next;
    }
}

static void free_queue(Queue *queue) {
    while (queue->front != NULL) {
        Customer *next = queue->front->next;
        free(queue->front);
        queue->front = next;
    }
    queue->rear = NULL;
}

int main(void) {
    Queue queue = {NULL, NULL};
    int choice;
    int tickets;
    char name[100];

    while (1) {
        printf("\nTicket Counter Menu\n");
        printf("1. Add reservation request\n");
        printf("2. Serve next customer\n");
        printf("3. Display queue\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
```

```
            printf("Enter customer name: ");
            read_line(name, sizeof(name));
            printf("Enter number of tickets: ");
            scanf("%d", &tickets);
            getchar();
            enqueue(&queue, name, tickets);
            break;
        case 2:
            serve_customer(&queue);
            break;
        case 3:
            display_queue(&queue);
            break;
        case 4:
            free_queue(&queue);
            return 0;
        default:
            printf("Invalid choice.\n");
    }
  }

  free_queue(&queue);
  return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./18_ticket_counter_queue

Ticket Counter Menu
1. Add reservation request
2. Serve next customer
3. Display queue
4. Exit
Enter your choice: 1
Enter customer name: Ashwin S
Enter number of tickets: 3

Ticket Counter Menu
1. Add reservation request
2. Serve next customer
3. Display queue
4. Exit
Enter your choice: 1
Enter customer name: John Doe
Enter number of tickets: 1

Ticket Counter Menu
1. Add reservation request
2. Serve next customer
3. Display queue
4. Exit
Enter your choice: 3
Customer: Ashwin S Tickets: 3
Customer: John Doe Tickets: 1

Ticket Counter Menu
1. Add reservation request
2. Serve next customer
3. Display queue
4. Exit
Enter your choice: 2
Booking 3 ticket(s) for Ashwin S

Ticket Counter Menu
1. Add reservation request
2. Serve next customer
3. Display queue
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the reservation system program was executed successfully. The booking process was performed correctly.

Question 19:

Write a C program to implement the task queue model.

Aim:

To write a C program to manage tasks using queue. Tasks are executed in arrival order.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Task {
    char name[100];
    int duration;
    struct Task *next;
} Task;

typedef struct {
    Task *front;
    Task *rear;
} Queue;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static void enqueue(Queue *queue, const char *name, int duration) {
    Task *task = (Task *) malloc(sizeof(Task));
    if (task == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(task->name, name, sizeof(task->name) - 1);
    task->name[sizeof(task->name) - 1] = '\0';
    task->duration = duration;
    task->next = NULL;

    if (queue->rear == NULL) {
        queue->front = queue->rear = task;
    } else {
        queue->rear->next = task;
        queue->rear = task;
    }
}

static void process_next(Queue *queue) {
    if (queue->front == NULL) {
        printf("Task queue is empty.\n");
        return;
```

```
    }

    Task *task = queue->front;
    printf("Processing task %s for %d unit(s)\n", task->name, task->duration);
    queue->front = task->next;
    if (queue->front == NULL) {
        queue->rear = NULL;
    }
    free(task);
}

static void display_queue(const Queue *queue) {
    if (queue->front == NULL) {
        printf("No tasks in queue.\n");
        return;
    }

    Task *temp = queue->front;
    while (temp != NULL) {
        printf("Task: %-15s Duration: %d\n", temp->name, temp->duration);
        temp = temp->next;
    }
}

static void free_queue(Queue *queue) {
    while (queue->front != NULL) {
        Task *next = queue->front->next;
        free(queue->front);
        queue->front = next;
    }
    queue->rear = NULL;
}

int main(void) {
    Queue queue = {NULL, NULL};
    int choice;
    int duration;
    char name[100];

    while (1) {
        printf("\nTask Queue Menu\n");
        printf("1. Add task\n");
        printf("2. Process next task\n");
        printf("3. Display task queue\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter task name: ");
                read_line(name, sizeof(name));
```

```
            printf("Enter task duration: ");
            scanf("%d", &duration);
            getchar();
            enqueue(&queue, name, duration);
            break;
        case 2:
            process_next(&queue);
            break;
        case 3:
            display_queue(&queue);
            break;
        case 4:
            free_queue(&queue);
            return 0;
        default:
            printf("Invalid choice.\n");
    }
}

free_queue(&queue);
return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./19_task_queue_model

Task Queue Menu
1. Add task
2. Process next task
3. Display task queue
4. Exit
Enter your choice: 1
Enter task name: Render Video
Enter task duration: 120

Task Queue Menu
1. Add task
2. Process next task
3. Display task queue
4. Exit
Enter your choice: 1
Enter task name: Download File
Enter task duration: 15

Task Queue Menu
1. Add task
2. Process next task
3. Display task queue
4. Exit
Enter your choice: 3
Task: Render Video Duration: 120
Task: Download File Duration: 15

Task Queue Menu
1. Add task
2. Process next task
3. Display task queue
4. Exit
Enter your choice: 2
Processing task Render Video for 120 unit(s)

Task Queue Menu
1. Add task
2. Process next task
3. Display task queue
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the task queue program was executed successfully. The tasks were processed correctly.

Question 20:

Write a C program to implement the shared resources allocation with fist come first serve basis model.

Aim:

To simulate shared resource allocation using FCFS scheduling. This ensures fair resource usage.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Request {
    char process_name[100];
    int requested_time;
    struct Request *next;
} Request;

typedef struct {
    Request *front;
    Request *rear;
} Queue;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Request *create_request(const char *name, int time) {
    Request *request = (Request *) malloc(sizeof(Request));
    if (request == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(request->process_name, name, sizeof(request->process_name) - 1);
    request->process_name[sizeof(request->process_name) - 1] = '\0';
    request->requested_time = time;
    request->next = NULL;
    return request;
}

static void enqueue(Queue *queue, Request *request) {
    if (queue->rear == NULL) {
        queue->front = queue->rear = request;
    } else {
        queue->rear->next = request;
        queue->rear = request;
    }
}

static Request *dequeue(Queue *queue) {
```

```
    if (queue->front == NULL) {
        return NULL;
    }

    Request *request = queue->front;
    queue->front = request->next;
    if (queue->front == NULL) {
        queue->rear = NULL;
    }
    request->next = NULL;
    return request;
}

static void display_queue(const Queue *queue, const Request *current) {
    if (current != NULL) {
        printf("Resource currently allocated to %s for %d unit(s)\n",
               current->process_name,
               current->requested_time);
    } else {
        printf("Resource is currently free.\n");
    }

    if (queue->front == NULL) {
        printf("No waiting requests.\n");
        return;
    }

    printf("Waiting queue:\n");
    Request *temp = queue->front;
    while (temp != NULL) {
        printf("Process: %-15s Time: %d\n", temp->process_name, temp->requested_time);
        temp = temp->next;
    }
}

static void free_queue(Queue *queue) {
    while (queue->front != NULL) {
        Request *next = queue->front->next;
        free(queue->front);
        queue->front = next;
    }
    queue->rear = NULL;
}

int main(void) {
    Queue queue = {NULL, NULL};
    Request *current = NULL;
    int choice;
    int time;
    char name[100];

    while (1) {
        printf("\nShared Resource FCFS Menu\n");
        printf("1. Request resource\n");
        printf("2. Release current resource holder\n");
```

```
        printf("3. Display allocation status\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter process name: ");
                read_line(name, sizeof(name));
                printf("Enter requested time units: ");
                scanf("%d", &time);
                getchar();
                if (current == NULL) {
                    current = create_request(name, time);
                    printf("Resource allocated immediately to %s\n", current->process_name);
                } else {
                    enqueue(&queue, create_request(name, time));
                    printf("Resource busy. Request added to waiting queue.\n");
                }
                break;
            case 2:
                if (current == NULL) {
                    printf("No process is holding the resource.\n");
                } else {
                    printf("Releasing resource from %s\n", current->process_name);
                    free(current);
                    current = dequeue(&queue);
                    if (current != NULL) {
                        printf("Resource allocated next to %s\n", current->process_name);
                    }
                }
                break;
            case 3:
                display_queue(&queue, current);
                break;
            case 4:
                free(current);
                free_queue(&queue);
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    free(current);
    free_queue(&queue);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./20_shared_resource_fcfs_queu

Shared Resource FCFS Menu
1. Request resource
2. Release current resource holder
3. Display allocation status
4. Exit
Enter your choice: 1
Enter process name: Job A
Enter requested time units: 15
Resource allocated immediately to Job A

Shared Resource FCFS Menu
1. Request resource
2. Release current resource holder
3. Display allocation status
4. Exit
Enter your choice: 1
Enter process name: Job B
Enter requested time units: 5
Resource busy. Request added to waiting queue.

Shared Resource FCFS Menu
1. Request resource
2. Release current resource holder
3. Display allocation status
4. Exit
Enter your choice: 3
Resource currently allocated to Job A for 15 unit(s)
Waiting queue:
Process: Job B Time: 5

Shared Resource FCFS Menu
1. Request resource
2. Release current resource holder
3. Display allocation status
4. Exit
Enter your choice: 2
Releasing resource from Job A
Resource allocated next to Job B

Shared Resource FCFS Menu
1. Request resource
2. Release current resource holder
3. Display allocation status
4. Exit
Enter your choice: 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the resource allocation program was executed successfully. The resources were assigned correctly.

Question 21:

Write a C program to implement message queue, buffered I/O and pipes model using queue

Aim:

To implement message passing system using queue. This program simulates buffered communication.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Message {
    char text[200];
    struct Message *next;
} Message;

typedef struct {
    Message *front;
    Message *rear;
} Queue;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Message *create_message(const char *text) {
    Message *message = (Message *) malloc(sizeof(Message));
    if (message == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(message->text, text, sizeof(message->text) - 1);
    message->text[sizeof(message->text) - 1] = '\0';
    message->next = NULL;
    return message;
}

static void enqueue(Queue *queue, Message *message) {
    if (queue->rear == NULL) {
        queue->front = queue->rear = message;
    } else {
        queue->rear->next = message;
        queue->rear = message;
    }
}

static Message *dequeue(Queue *queue) {
    if (queue->front == NULL) {
```

```
        return NULL;
    }

    Message *message = queue->front;
    queue->front = message->next;
    if (queue->front == NULL) {
        queue->rear = NULL;
    }
    message->next = NULL;
    return message;
}

static void display_queue(const char *label, const Queue *queue) {
    printf("%s:\n", label);
    if (queue->front == NULL) {
        printf("  Empty\n");
        return;
    }

    Message *temp = queue->front;
    while (temp != NULL) {
        printf("  %s\n", temp->text);
        temp = temp->next;
    }
}

static void free_queue(Queue *queue) {
    while (queue->front != NULL) {
        Message *next = queue->front->next;
        free(queue->front);
        queue->front = next;
    }
    queue->rear = NULL;
}

int main(void) {
    Queue message_queue = {NULL, NULL};
    Queue pipe_buffer = {NULL, NULL};
    int choice;
    char text[200];

    while (1) {
        printf("\nMessage Queue and Pipe Buffer Menu\n");
        printf("1. Send message to message queue\n");
        printf("2. Move one message to pipe buffer\n");
        printf("3. Receive message from pipe buffer\n");
        printf("4. Display queues\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
```

```
        case 1:
            printf("Enter message text: ");
            read_line(text, sizeof(text));
            enqueue(&message_queue, create_message(text));
            break;
        case 2:
            {
                Message *message = dequeue(&message_queue);
                if (message == NULL) {
                    printf("Message queue is empty.\n");
                } else {
                    enqueue(&pipe_buffer, message);
                    printf("Message moved into pipe buffer.\n");
                }
            }
            break;
        case 3:
            {
                Message *message = dequeue(&pipe_buffer);
                if (message == NULL) {
                    printf("Pipe buffer is empty.\n");
                } else {
                    printf("Consumer received: %s\n", message->text);
                    free(message);
                }
            }
            break;
        case 4:
            display_queue("Message queue", &message_queue);
            display_queue("Pipe buffer", &pipe_buffer);
            break;
        case 5:
            free_queue(&message_queue);
            free_queue(&pipe_buffer);
            return 0;
        default:
            printf("Invalid choice.\n");
    }
}

free_queue(&message_queue);
free_queue(&pipe_buffer);
return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./21_message_queue_buffered_io

Message Queue and Pipe Buffer Menu
1. Send message to message queue
2. Move one message to pipe buffer
3. Receive message from pipe buffer
4. Display queues
5. Exit
Enter your choice: 1
Enter message text: Hello from Producer

Message Queue and Pipe Buffer Menu
1. Send message to message queue
2. Move one message to pipe buffer
3. Receive message from pipe buffer
4. Display queues
5. Exit
Enter your choice: 2
Message moved into pipe buffer.

Message Queue and Pipe Buffer Menu
1. Send message to message queue
2. Move one message to pipe buffer
3. Receive message from pipe buffer
4. Display queues
5. Exit
Enter your choice: 4
Message queue:
Empty
Pipe buffer:
Hello from Producer

Message Queue and Pipe Buffer Menu
1. Send message to message queue
2. Move one message to pipe buffer
3. Receive message from pipe buffer
4. Display queues
5. Exit
Enter your choice: 3
Consumer received: Hello from Producer

Message Queue and Pipe Buffer Menu
1. Send message to message queue
2. Move one message to pipe buffer
3. Receive message from pipe buffer
4. Display queues
5. Exit
Enter your choice: 5
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the message queue program was executed successfully. The messages were transmitted correctly.

Question 22:

Write a C program to read the table from the NAND Flash, perform selection sort and rearrange the names in ascending order and store the table back to the NAND Flash. Note: When the write operations are extremely expensive then the read operations, then selection sort could be ideal.

Aim:

To sort data stored in memory using selection sort algorithm. This reduces write operations.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 100
#define MAX_NAME 100

static void swap_names(char names[][MAX_NAME], int i, int j) {
    char temp[MAX_NAME];
    strcpy(temp, names[i]);
    strcpy(names[i], names[j]);
    strcpy(names[j], temp);
}

int main(void) {
    int n;
    char names[MAX_ITEMS][MAX_NAME];

    printf("Enter number of names in NAND Flash table: ");
    scanf("%d", &n);
    getchar();

    if (n <= 0 || n > MAX_ITEMS) {
        printf("Invalid number of entries.\n");
        return 0;
    }

    printf("Enter %d names:\n", n);
    for (int i = 0; i < n; ++i) {
        fgets(names[i], sizeof(names[i]), stdin);
        names[i][strcspn(names[i], "\n")] = '\0';
    }

    printf("\nOriginal table:\n");
    for (int i = 0; i < n; ++i) {
        printf("%s\n", names[i]);
    }

    for (int i = 0; i < n - 1; ++i) {
        int min_index = i;
        for (int j = i + 1; j < n; ++j) {
            if (strcmp(names[j], names[min_index]) < 0) {
                min_index = j;
            }
        }
        if (min_index != i) {
```

```
            swap_names(names, i, min_index);
        }
    }

    printf("\nRearranged table stored back in ascending order:\n");
    for (int i = 0; i < n; ++i) {
        printf("%s\n", names[i]);
    }

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./22_selection_sort_nand_flash
Enter number of names in NAND Flash table: 4
Enter 4 names:
Charlie
Alice
Delta
Bob

Original table:
Charlie
Alice
Delta
Bob

Rearranged table stored back in ascending order:
Alice
Bob
Charlie
Delta
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the selection sort program was executed successfully. The data was arranged in ascending order.

Question 23:
Write a C program to perform bubble sort whenever new flight arrival details are added in the arrival table.

Aim:
To maintain sorted arrival details using bubble sort. This program updates sorted list dynamically.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_FLIGHTS 100

typedef struct {
    char flight_number[20];
    int arrival_time;
} Flight;

static void bubble_sort(Flight flights[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (flights[j].arrival_time > flights[j + 1].arrival_time) {
                Flight temp = flights[j];
                flights[j] = flights[j + 1];
                flights[j + 1] = temp;
            }
        }
    }
}

static void display_flights(const Flight flights[], int n) {
    if (n == 0) {
        printf("Arrival table is empty.\n");
        return;
    }

    printf("\nArrival table:\n");
    for (int i = 0; i < n; ++i) {
        printf("Flight: %-10s Arrival time: %04d\n", flights[i].flight_number, flights[i].arrival_time);
    }
}

int main(void) {
    Flight flights[MAX_FLIGHTS];
    int count = 0;
    int choice;

    while (1) {
        printf("\nFlight Arrival Menu\n");
        printf("1. Add new flight arrival\n");
        printf("2. Display sorted arrival table\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
```

```
        }

        switch (choice) {
            case 1:
                if (count >= MAX_FLIGHTS) {
                    printf("Arrival table is full.\n");
                    break;
                }
                printf("Enter flight number: ");
                scanf("%19s", flights[count].flight_number);
                printf("Enter arrival time in HHMM format: ");
                scanf("%d", &flights[count].arrival_time);
                ++count;
                bubble_sort(flights, count);
                printf("Flight added and table sorted using bubble sort.\n");
                break;
            case 2:
                display_flights(flights, count);
                break;
            case 3:
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./23_bubble_sort_flight_arriva

Flight Arrival Menu
1. Add new flight arrival
2. Display sorted arrival table
3. Exit
Enter your choice: 1
Enter flight number: AI802
Enter arrival time in HHMM format: 1430
Flight added and table sorted using bubble sort.

Flight Arrival Menu
1. Add new flight arrival
2. Display sorted arrival table
3. Exit
Enter your choice: 1
Enter flight number: EK501
Enter arrival time in HHMM format: 0915
Flight added and table sorted using bubble sort.

Flight Arrival Menu
1. Add new flight arrival
2. Display sorted arrival table
3. Exit
Enter your choice: 2

Arrival table:
Flight: EK501 Arrival time: 0915
Flight: AI802 Arrival time: 1430

Flight Arrival Menu
1. Add new flight arrival
2. Display sorted arrival table
3. Exit
Enter your choice: 3
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the bubble sort program was executed successfully. The flight records were arranged correctly.

Question 24:

Write a C program to sort the media file according to the file names using insertion sorting.

Aim:

To sort media files based on file names using insertion sort. This program maintains ordered list.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_FILES 100
#define MAX_NAME 100

int main(void) {
    int n;
    char files[MAX_FILES][MAX_NAME];
    char key[MAX_NAME];

    printf("Enter number of media files: ");
    scanf("%d", &n);
    getchar();

    if (n <= 0 || n > MAX_FILES) {
        printf("Invalid number of files.\n");
        return 0;
    }

    printf("Enter the file names:\n");
    for (int i = 0; i < n; ++i) {
        fgets(files[i], sizeof(files[i]), stdin);
        files[i][strcspn(files[i], "\n")] = '\0';
    }

    for (int i = 1; i < n; ++i) {
        strcpy(key, files[i]);
        int j = i - 1;
        while (j >= 0 && strcmp(files[j], key) > 0) {
            strcpy(files[j + 1], files[j]);
            --j;
        }
        strcpy(files[j + 1], key);
    }

    printf("\nMedia files sorted using insertion sort:\n");
    for (int i = 0; i < n; ++i) {
        printf("%s\n", files[i]);
    }

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./24_insertion_sort_media_fil
Enter number of media files: 4
Enter the file names:
video_trip.mp4
audio_track.mp3
photo_vacation.png
document_notes.pdf

Media files sorted using insertion sort:
audio_track.mp3
document_notes.pdf
photo_vacation.png
video_trip.mp4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the insertion sort program was executed successfully. The file names were sorted correct

Question 25:

Write a C program to sort the product list according to their price using merge sorting.

Aim:

To implement merge sort to arrange products based on price. This demonstrates divide and conquer method.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_PRODUCTS 100

typedef struct {
    char name[100];
    float price;
} Product;

static void merge(Product products[], int left, int mid, int right) {
    Product temp[MAX_PRODUCTS];
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right) {
        if (products[i].price <= products[j].price) {
            temp[k++] = products[i++];
        } else {
            temp[k++] = products[j++];
        }
    }

    while (i <= mid) {
        temp[k++] = products[i++];
    }

    while (j <= right) {
        temp[k++] = products[j++];
    }

    for (i = left; i <= right; ++i) {
        products[i] = temp[i];
    }
}

static void merge_sort(Product products[], int left, int right) {
    if (left >= right) {
        return;
    }

    int mid = left + (right - left) / 2;
    merge_sort(products, left, mid);
    merge_sort(products, mid + 1, right);
    merge(products, left, mid, right);
```

```
}

int main(void) {
    Product products[MAX_PRODUCTS];
    int n;

    printf("Enter number of products: ");
    scanf("%d", &n);
    getchar();

    if (n <= 0 || n > MAX_PRODUCTS) {
        printf("Invalid number of products.\n");
        return 0;
    }

    for (int i = 0; i < n; ++i) {
        printf("Enter product %d name: ", i + 1);
        fgets(products[i].name, sizeof(products[i].name), stdin);
        products[i].name[strcspn(products[i].name, "\n")] = '\0';
        printf("Enter product %d price: ", i + 1);
        scanf("%f", &products[i].price);
        getchar();}
    merge_sort(products, 0, n - 1);

    printf("\nProducts sorted by price using merge sort:\n");
    for (int i = 0; i < n; ++i) {
        printf("%-20s %.2f\n", products[i].name, products[i].price); }
    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./25_merge_sort_products_by_p
Enter number of products: 3
Enter product 1 name: Laptop
Enter product 1 price: 1200.50
Enter product 2 name: Mouse
Enter product 2 price: 25.99
Enter product 3 name: Keyboard
Enter product 3 price: 45.00

Products sorted by price using merge sort:
Mouse 25.99
Keyboard 45.00
Laptop 1200.50
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the merge sort program was executed successfully. The product list was sorted correctly.

Question 26:

Write a C program to sort the hazardous or dangerous material / chemicals in the lab using quick sorting.

Aim:

To sort dangerous materials using quick sort algorithm. This program ensures efficient arrangement.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_CHEMICALS 100

typedef struct {
    char name[100];
    int hazard_level;
} Chemical;

static void swap(Chemical *a, Chemical *b) {
    Chemical temp = *a;
    *a = *b;
    *b = temp;
}

static int partition(Chemical chemicals[], int low, int high) {
    int pivot = chemicals[high].hazard_level;
    int i = low - 1;

    for (int j = low; j < high; ++j) {
        if (chemicals[j].hazard_level >= pivot) {
            ++i;
            swap(&chemicals[i], &chemicals[j]);
        }
    }

    swap(&chemicals[i + 1], &chemicals[high]);
    return i + 1;
}

static void quick_sort(Chemical chemicals[], int low, int high) {
    if (low < high) {
        int pivot_index = partition(chemicals, low, high);
        quick_sort(chemicals, low, pivot_index - 1);
        quick_sort(chemicals, pivot_index + 1, high);
    }
}

int main(void) {
    Chemical chemicals[MAX_CHEMICALS];
    int n;

    printf("Enter number of hazardous materials: ");
    scanf("%d", &n);
    getchar();
```

```
    if (n <= 0 || n > MAX_CHEMICALS) {
        printf("Invalid number of materials.\n");
        return 0;
    }

    for (int i = 0; i < n; ++i) {
        printf("Enter chemical %d name: ", i + 1);
        fgets(chemicals[i].name, sizeof(chemicals[i].name), stdin);
        chemicals[i].name[strcspn(chemicals[i].name, "\n")] = '\0';
        printf("Enter hazard level for %s: ", chemicals[i].name);
        scanf("%d", &chemicals[i].hazard_level);
        getchar();
    }

    quick_sort(chemicals, 0, n - 1);

    printf("\nChemicals sorted by hazard level using quick sort:\n");
    for (int i = 0; i < n; ++i) {
        printf("%-20s Hazard Level: %d\n", chemicals[i].name, chemicals[i].hazard_level);
    }

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./26_quick_sort_hazardous_mat
Enter number of hazardous materials: 3
Enter chemical 1 name: Plutonium
Enter hazard level for Plutonium: 9
Enter chemical 2 name: Bleach
Enter hazard level for Bleach: 3
Enter chemical 3 name: Acid
Enter hazard level for Acid: 7

Chemicals sorted by hazard level using quick sort:
Plutonium Hazard Level: 9
Acid Hazard Level: 7
Bleach Hazard Level: 3
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the quick sort program was executed successfully. The materials were arranged correctly.

Question 27:

Write a C program to rearrange the task list based on the task priority.

Aim:

To rearrange tasks according to their priority values. This helps to execute important tasks first.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Task {
    char name[100];
    int priority;
    struct Task *next;
} Task;

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int) size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
}

static Task *create_task(const char *name, int priority) {
    Task *task = (Task *) malloc(sizeof(Task));
    if (task == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    strncpy(task->name, name, sizeof(task->name) - 1);
    task->name[sizeof(task->name) - 1] = '\0';
    task->priority = priority;
    task->next = NULL;
    return task;
}

static void insert_by_priority(Task **head, const char *name, int priority) {
    Task *task = create_task(name, priority);

    if (*head == NULL || priority > (*head)->priority) {
        task->next = *head;
        *head = task;
        return;
    }

    Task *temp = *head;
    while (temp->next != NULL && temp->next->priority >= priority) {
        temp = temp->next;
    }
    task->next = temp->next;
    temp->next = task;
}
```

```
static void display_tasks(Task *head) {
    if (head == NULL) {
        printf("No tasks available.\n");
        return;
    }

    printf("\nTask list rearranged by priority:\n");
    while (head != NULL) {
        printf("Task: %-15s Priority: %d\n", head->name, head->priority);
        head = head->next;
    }
}

static void free_tasks(Task *head) {
    while (head != NULL) {
        Task *next = head->next;
        free(head);
        head = next;
    }
}

int main(void) {
    Task *head = NULL;
    int choice;
    int priority;
    char name[100];

    while (1) {
        printf("\nTask Priority Menu\n");
        printf("1. Add task\n");
        printf("2. Display rearranged task list\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }
        getchar();

        switch (choice) {
            case 1:
                printf("Enter task name: ");
                read_line(name, sizeof(name));
                printf("Enter task priority: ");
                scanf("%d", &priority);
                getchar();
                insert_by_priority(&head, name, priority);
                break;
            case 2:
                display_tasks(head);
                break;
            case 3:
                free_tasks(head);
                return 0;
            default:
                printf("Invalid choice.\n");
```

```
        }
    }

    free_tasks(head);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./27_task_priority_rearrangeme

Task Priority Menu
1. Add task
2. Display rearranged task list
3. Exit
Enter your choice: 1
Enter task name: LowPriorityTask
Enter task priority: 1

Task Priority Menu
1. Add task
2. Display rearranged task list
3. Exit
Enter your choice: 1
Enter task name: HighPriorityTask
Enter task priority: 10

Task Priority Menu
1. Add task
2. Display rearranged task list
3. Exit
Enter your choice: 2

Task list rearranged by priority:
Task: HighPriorityTask Priority: 10
Task: LowPriorityTask Priority: 1

Task Priority Menu
1. Add task
2. Display rearranged task list
3. Exit
Enter your choice: 3
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the task priority program was executed successfully. The tasks were reordered correctly.

Question 28:

Write a C program to implement priority-based scheduling using heap sort.

Aim:

To implement priority based scheduling using heap sort. This program organizes tasks efficiently.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_PROCESSES 100

typedef struct {
    char name[100];
    int priority;
    int burst_time;
} Process;

static void swap(Process *a, Process *b) {
    Process temp = *a;
    *a = *b;
    *b = temp;
}

static void heapify(Process processes[], int n, int index) {
    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < n && processes[left].priority > processes[largest].priority) {
        largest = left;
    }
    if (right < n && processes[right].priority > processes[largest].priority) {
        largest = right;
    }

    if (largest != index) {
        swap(&processes[index], &processes[largest]);
        heapify(processes, n, largest);
    }
}

static void heap_sort(Process processes[], int n) {
    for (int i = n / 2 - 1; i >= 0; --i) {
        heapify(processes, n, i);
    }

    for (int i = n - 1; i > 0; --i) {
        swap(&processes[0], &processes[i]);
        heapify(processes, i, 0);
    }
}

int main(void) {
    Process processes[MAX_PROCESSES];
```

```
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    getchar();

    if (n <= 0 || n > MAX_PROCESSES) {
        printf("Invalid number of processes.\n");
        return 0;
    }

    for (int i = 0; i < n; ++i) {
        printf("Enter process %d name: ", i + 1);
        fgets(processes[i].name, sizeof(processes[i].name), stdin);
        processes[i].name[strcspn(processes[i].name, "\n")] = '\0';
        printf("Enter priority for %s: ", processes[i].name);
        scanf("%d", &processes[i].priority);
        printf("Enter burst time for %s: ", processes[i].name);
        scanf("%d", &processes[i].burst_time);
        getchar(); }
    heap_sort(processes, n);
    printf("\nPriority-based scheduling order using heap sort:\n");
    for (int i = n - 1; i >= 0; --i) {
        printf("Process: %-15s Priority: %-4d Burst: %d\n",
               processes[i].name, processes[i].priority, processes[i].burst_time); }
    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./28_priority_scheduling_heap_
Enter number of processes: 3
Enter process 1 name: P1
Enter priority for P1: 2
Enter burst time for P1: 10
Enter process 2 name: P2
Enter priority for P2: 5
Enter burst time for P2: 5
Enter process 3 name: P3
Enter priority for P3: 1
Enter burst time for P3: 20

Priority-based scheduling order using heap sort:
Process: P2 Priority: 5 Burst: 5
Process: P1 Priority: 2 Burst: 10
Process: P3 Priority: 1 Burst: 20
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the heap sort scheduling program was executed successfully. The priority order was maintained.

Question 29:

Write a C program to find the most frequent used words from a file

Aim:

To develop a C program to identify most repeated words in a file. This helps in text analysis.

Code:

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_WORDS 5000
#define MAX_WORD_LEN 64

typedef struct {
    char word[MAX_WORD_LEN];
    int count;
} WordEntry;

static int compare_frequency(const void *a, const void *b) {
    const WordEntry *left = (const WordEntry *) a;
    const WordEntry *right = (const WordEntry *) b;

    if (right->count != left->count) {
        return right->count - left->count;
    }
    return strcmp(left->word, right->word);
}

static void add_word(WordEntry entries[], int *size, const char *word) {
    for (int i = 0; i < *size; ++i) {
        if (strcmp(entries[i].word, word) == 0) {
            ++entries[i].count;
            return;
        }
    }

    if (*size >= MAX_WORDS) {
        return;
    }

    strncpy(entries[*size].word, word, MAX_WORD_LEN - 1);
    entries[*size].word[MAX_WORD_LEN - 1] = '\0';
    entries[*size].count = 1;
    ++(*size);
}

int main(void) {
    char path[256];
    FILE *file;
    WordEntry entries[MAX_WORDS];
    int entry_count = 0;
    char word[MAX_WORD_LEN];
    int length = 0;
```

```
    int ch;

    printf("Enter file path: ");
    if (fgets(path, sizeof(path), stdin) == NULL) {
        return 0;
    }
    path[strcspn(path, "\n")] = '\0';

    file = fopen(path, "r");
    if (file == NULL) {
        printf("Unable to open file.\n");
        return 0;
    }

    while ((ch = fgetc(file)) != EOF) {
        if (isalnum((unsigned char) ch)) {
            if (length < MAX_WORD_LEN - 1) {
                word[length++] = (char) tolower((unsigned char) ch);
            }
        } else if (length > 0) {
            word[length] = '\0';
            add_word(entries, &entry_count, word);
            length = 0;
        }
    }

    if (length > 0) {
        word[length] = '\0';
        add_word(entries, &entry_count, word);
    }

    fclose(file);

    if (entry_count == 0) {
        printf("No words found in the file.\n");
        return 0;
    }

    qsort(entries, (size_t) entry_count, sizeof(entries[0]), compare_frequency);

    printf("\nMost frequent words:\n");
    int limit = entry_count < 10 ? entry_count : 10;
    for (int i = 0; i < limit; ++i) {
        printf("%-20s %d\n", entries[i].word, entries[i].count);
    }

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./29_most_frequent_words_from_
Enter file path: sample_text.txt

Most frequent words:
the 15
data 12
algorithm 8
system 6
file 4
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the word frequency program was executed successfully. The frequent words were found correctly.

Question 30:

Write a C program to use Pthread for binary search to identify the specific name.

Aim:

To search a specific book from large collection using efficient searching method. This helps in fast retrieval.

Code:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_NAME 100

typedef struct {
    char (*names)[MAX_NAME];
    int count;
    char target[MAX_NAME];
    int result;
} SearchArgs;

static int compare_names(const void *a, const void *b) {
    const char *left = (const char *) a;
    const char *right = (const char *) b;
    return strcmp(left, right);
}

static void *binary_search_thread(void *arg) {
    SearchArgs *args = (SearchArgs *) arg;
    int left = 0;
    int right = args->count - 1;
    args->result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int compare = strcmp(args->target, args->names[mid]);

        if (compare == 0) {
            args->result = mid;
            break;
        }
        if (compare < 0) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    return NULL;
}

int main(void) {
    int n;
```

```
    pthread_t thread;
    SearchArgs args = {NULL, 0, "", -1};

    printf("Enter number of names: ");
    scanf("%d", &n);
    getchar();

    if (n <= 0) {
        printf("Invalid number of names.\n");
        return 0;
    }

    args.names = malloc((size_t) n * sizeof(*args.names));
    if (args.names == NULL) {
        printf("Memory allocation failed.\n");
        return 0;
    }
    args.count = n;

    printf("Enter %d names:\n", n);
    for (int i = 0; i < n; ++i) {
        fgets(args.names[i], MAX_NAME, stdin);
        args.names[i][strcspn(args.names[i], "\n")] = '\0';
    }

    qsort(args.names, (size_t) n, sizeof(*args.names), compare_names);

    printf("Enter name to search: ");
    fgets(args.target, sizeof(args.target), stdin);
    args.target[strcspn(args.target, "\n")] = '\0';

    if (pthread_create(&thread, NULL, binary_search_thread, &args) != 0) {
        printf("Unable to create thread.\n");
        free(args.names);
        return 0;
    }

    pthread_join(thread, NULL);

    printf("\nSorted names:\n");
    for (int i = 0; i < n; ++i) {
        printf("%d. %s\n", i, args.names[i]);
    }

    if (args.result >= 0) {
        printf("Name found at sorted index %d\n", args.result);
    } else {
        printf("Name not found.\n");
    }

    free(args.names);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./30_pthread_binary_search_nam
Enter number of names: 4
Enter 4 names:
Zeta
Beta
Alpha
Gamma
Enter name to search: Gamma

Sorted names:
0. Alpha
1. Beta
2. Gamma
3. Zeta
Name found at sorted index 2
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the library search program was executed successfully. The required book details were obtained correctly.

Question 31:

In a library, there are thousands of books to go through. Write a C program to to find the exact book, implement search program.

Aim:

To search a specific book from large collection using efficient searching method. This helps in fast retrieval.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_BOOKS 1000

typedef struct {
    char title[100];
    char author[100];
    int id;
} Book;

static int compare_books(const void *a, const void *b) {
    const Book *left = (const Book *) a;
    const Book *right = (const Book *) b;
    return strcmp(left->title, right->title);
}

static int binary_search_books(const Book books[], int n, const char *title) {
    int left = 0;
    int right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int compare = strcmp(title, books[mid].title);

        if (compare == 0) {
            return mid;
        }
        if (compare < 0) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    return -1;
}

int main(void) {
    Book books[MAX_BOOKS];
    int n;
    char target[100];

    printf("Enter number of books: ");
```

```
    scanf("%d", &n);
    getchar();

    if (n <= 0 || n > MAX_BOOKS) {
        printf("Invalid number of books.\n");
        return 0;
    }

    for (int i = 0; i < n; ++i) {
        printf("Enter title of book %d: ", i + 1);
        fgets(books[i].title, sizeof(books[i].title), stdin);
        books[i].title[strcspn(books[i].title, "\n")] = '\0';
        printf("Enter author of %s: ", books[i].title);
        fgets(books[i].author, sizeof(books[i].author), stdin);
        books[i].author[strcspn(books[i].author, "\n")] = '\0';
        printf("Enter ID of %s: ", books[i].title);
        scanf("%d", &books[i].id);
        getchar();
    }

    qsort(books, (size_t) n, sizeof(books[0]), compare_books);

    printf("Enter exact book title to search: ");
    fgets(target, sizeof(target), stdin);
    target[strcspn(target, "\n")] = '\0';

    int index = binary_search_books(books, n, target);
    if (index >= 0) {
        printf("Book found.\n");
        printf("Title : %s\n", books[index].title);
        printf("Author: %s\n", books[index].author);
        printf("ID    : %d\n", books[index].id);
    } else {
        printf("Book not found.\n");
    }

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./31_library_book_search
Enter number of books: 2
Enter title of book 1: Data Structures
Enter author of Data Structures: Mark Allen Weiss
Enter ID of Data Structures: 101
Enter title of book 2: Algorithms
Enter author of Algorithms: Thomas Cormen
Enter ID of Algorithms: 102
Enter exact book title to search: Algorithms
Book found.
Title : Algorithms
Author: Thomas Cormen
ID : 102
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the library search program was executed successfully. The required book details were obtained correctly.

Question 32:

Write a C program to use binary search to find out the file or directory by giving the name.

Aim:

To search a file or directory name using binary search technique. This reduces searching time.

Code:

```
#include <dirent.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#ifndef PATH_MAX
#define PATH_MAX 4096
#endif

typedef struct {
    char name[256];
    int is_directory;
} Entry;

static int compare_entries(const void *a, const void *b) {
    const Entry *left = (const Entry *) a;
    const Entry *right = (const Entry *) b;
    return strcmp(left->name, right->name);
}

static int binary_search_entries(const Entry entries[], int count, const char *target) {
    int left = 0;
    int right = count - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int compare = strcmp(target, entries[mid].name);

        if (compare == 0) {
            return mid;
        }
        if (compare < 0) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    return -1;
}

int main(void) {
    char directory_path[PATH_MAX];
    char target[256];
    DIR *directory;
```

```
struct dirent *record;
Entry *entries = NULL;
int count = 0;
int capacity = 0;

printf("Enter directory path to search in: ");
if (fgets(directory_path, sizeof(directory_path), stdin) == NULL) {
    return 0;
}
directory_path[strcspn(directory_path, "\n")] = '\0';

directory = opendir(directory_path);
if (directory == NULL) {
    printf("Unable to open directory.\n");
    return 0;
}

while ((record = readdir(directory)) != NULL) {
    if (strcmp(record->d_name, ".") == 0 || strcmp(record->d_name, "..") == 0) {
        continue;
    }

    if (count == capacity) {
        capacity = capacity == 0 ? 16 : capacity * 2;
        Entry *new_entries = realloc(entries, (size_t) capacity * sizeof(*entries));
        if (new_entries == NULL) {
            free(entries);
            closedir(directory);
            printf("Memory allocation failed.\n");
            return 0;
        }
        entries = new_entries;
    }

    snprintf(entries[count].name, sizeof(entries[count].name), "%s", record->d_name);

    char full_path[PATH_MAX];
    struct stat path_info;
    size_t dir_len = strlen(directory_path);
    size_t name_len = strlen(record->d_name);

    if (dir_len + 1 + name_len < sizeof(full_path)) {
        memcpy(full_path, directory_path, dir_len);
        full_path[dir_len] = '/';
        memcpy(full_path + dir_len + 1, record->d_name, name_len + 1);

        if (stat(full_path, &path_info) == 0 && S_ISDIR(path_info.st_mode)) {
            entries[count].is_directory = 1;
        } else {
            entries[count].is_directory = 0;
        }
    } else {
        entries[count].is_directory = 0;
    }
```

```
        ++count;}
    closedir(directory);
    if (count == 0) {
        free(entries);
        printf("No files or directories found.\n");
        return 0;    }
    qsort(entries, (size_t) count, sizeof(*entries), compare_entries);
    printf("Enter file or directory name to search: ");
    fgets(target, sizeof(target), stdin);
    target[strcspn(target, "\n")] = '\0';
    int index = binary_search_entries(entries, count, target);
    if (index >= 0) {
        printf("%s found. Type: %s\n",
              entries[index].name,
              entries[index].is_directory ? "Directory" : "File");
    } else {
        printf("Entry not found.\n");}
    free(entries);
    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$ ./32_binary_search_file_or_dir
Enter directory path to search in: .
Enter file or directory name to search: 1_audio_player.c
1_audio_player.c found. Type: File
(Ashwin S 717823E206) silicondoc@arch Ass_2_ds$
```

Result:

Thus the binary search program was executed successfully. The file or directory was located correctly.

PREPARATION	**30**	
LAB PERFORMANCE	**30**	
REPORT	**40**	
TOTAL	**100**	
INITIAL OF FACULTY		